

README

July 15, 2026

1 Project Description

This package contains the numerical codes and example files used to reproduce the results presented in the manuscript.

“A GPU-Accelerated Dimensionless Framework for Dynamic Modeling of Multi-Scale Fluid Flow in Porous Media.”

2 File List

File	Description
<code>main.m</code>	Main MATLAB script.
<code>init_data3D_v25.m</code>	Pre-processing script for domain decomposition for MPI.
<code>MPI_EV_ZY.cu</code>	CUDA-MPI solver for 3-D wave propagation in pore-scale models.
<code>compute_Vp_Q_from_Vx.m</code>	Post-processing routine for computing velocity and attenuation.
<code>geocomp_unil_mpi3D_v4.m</code>	MPI configuration module.

3 Software Requirements

The following software is required:

- MATLAB R2022a or later
- CUDA Toolkit 12.0 or later
- OpenMPI 4.1.8 or later
- GCC 8.5.0 or later
- Linux operating system

4 Running the Code

Step 1: Pre-processing

1. Model Generation

Construct a numerical model represented by a label matrix. Each grid cell is assigned an integer label corresponding to a specific material or geometric feature. For example, a value of 0 may represent fractures, while a value of 1 represents the background medium. Additional labels can be defined for other materials or fluid-saturation regions as needed.

2. Build Material Property Matrices and Partition the Domain for MPI

Before running the simulation, specify the number of GPUs to be used. The computational domain is then automatically partitioned into subdomains according to the GPU configuration for MPI-based parallel execution in **main.m**.

```
dims_x = 4;
dims_y = 1;
dims_z = 1;

if mod(nx, dims_x) ~= 0 || mod(ny, dims_y) ~= 0 || mod(nz, dims_z) ~= 0
    error('The global grid must be divisible by the MPI decomposition.');
```



```
end

nx_l = nx / dims_x;
ny_l = ny / dims_y;
nz_l = nz / dims_z;

OVERLENGTH_X = 2 * (dims_x - 1);
OVERLENGTH_Y = 2 * (dims_y - 1);
OVERLENGTH_Z = 2 * (dims_z - 1);

nx_write = nx - OVERLENGTH_X;
ny_write = ny - OVERLENGTH_Y;
nz_write = nz - OVERLENGTH_Z;

if nx_write <= 0 || ny_write <= 0 || nz_write <= 0
    error('Invalid overlap-adjusted model dimensions.');
```



```
end

machineformat = 'n';
PRECIS = 8;

Total_GPU = dims_x * dims_y * dims_z;
GPU_x = dims_x;
GPU_y = dims_y;
GPU_z = dims_z;

NBX = fix(nx_l / 32);
NBY = fix(ny_l / 4);
NBZ = fix(nz_l / 8);
```

In this example, the computational domain is divided into four subdomains along the (x)-direction, resulting in a total of four GPUs being used for the simulation. After assigning the material properties and applying the dimensionless scaling parameters, the resulting property matrices are partitioned into (n) MPI subdomains (here ($n = \text{dims_x}$)). Each subdomain is subsequently written to a separate binary file for distributed GPU computation. The partitioning is performed using the following commands in **main.m**:

```

init_data3D_v25( ...
    G(1:nx_write, 1:ny_write, 1:nz_write), ...
    'G', 0, nx_l, ny_l, nz_l, ...
    dims_x, dims_y, dims_z, machineformat, PRECIS);

init_data3D_v25( ...
    K(1:nx_write, 1:ny_write, 1:nz_write), ...
    'K', 0, nx_l, ny_l, nz_l, ...
    dims_x, dims_y, dims_z, machineformat, PRECIS);

init_data3D_v25( ...
    rho(1:nx_write, 1:ny_write, 1:nz_write), ...
    'rho', 0, nx_l, ny_l, nz_l, ...
    dims_x, dims_y, dims_z, machineformat, PRECIS);

init_data3D_v25( ...
    volumeData(1:nx_write, 1:ny_write, 1:nz_write), ...
    'mask', 0, nx_l, ny_l, nz_l, ...
    dims_x, dims_y, dims_z, machineformat, PRECIS);

```

More details about MPI for numerical simulations of poroelastic wave can be found in the paper:

“Resolving Wave Propagation in Anisotropic Poroelastic Media Using Graphical Processing Units (GPUs).”

Step 2: Compilation and Execution

After generating the model and partitioning the domain, compile and launch the CUDA-MPI solver from **main.m** using

```

%% -----
% Run CUDA-MPI solver
%% -----

pathToScript = fullfile(pwd, 'runme_volta.sh');

Bash_run = [pathToScript, ' ', ...
int2str(Total_GPU), ' ', ...
int2str(GPU_x), ' ', ...
int2str(GPU_y), ' ', ...
int2str(GPU_z), ' ', ...
int2str(length(pa1)), ' ', ...
int2str(NBX), ' ', ...
int2str(NBY), ' ', ...
int2str(NBZ)];

fprintf('Running command:\n%s\n', Bash_run);

system(Bash_run);

```

This command executes the shell script **runme_volta.sh**, which compiles and launches the CUDA-MPI solver with the specified MPI decomposition and GPU configuration.

The compilation command used in **runme_volta.sh** is

```

nvcc -arch=sm_70
--compiler-bindir mpic++

```

```
--compiler-options -O3
MPI_Biot_ZY.cu
-DD_x=$2 -DD_y=$3 -DD_z=$4
-DNPARS1=$5
-DNBX=$6 -DNBY=$7 -DNBZ=$8
```

The flag `-arch=sm_70` is used for NVIDIA V100 GPUs. If a different GPU architecture is used, this flag should be modified accordingly. For example:

- V100: `sm_70`
- A100: `sm_80`
- H100: `sm_90`
- RTX 3090: `sm_86`

Users should update the GPU architecture flag in `runme_volta.sh` to match their hardware before compilation.

Important Notes

To reduce GPU memory consumption, the receiver responses are averaged over the receiver plane during the simulation, rather than storing the full wavefield at every receiver location. This significantly decreases the memory requirements and enables larger-scale simulations on GPUs with limited memory.

Compatibility with Older GPU Architectures

The code uses double-precision `atomicAdd()`, which is natively supported only on NVIDIA GPUs with compute capability 6.0 or higher (Pascal architecture and newer).

When compiling the code for older GPU architectures (`sm_52`, `sm_50`, etc.), compilation may fail with an error related to double-precision atomic operations. In this case, please copy the following code into `MPI_Biot_ZY.cu` immediately after

```
#include "time.h"
```

and remove the surrounding comment symbols:

```
#if defined(**CUDA_ARCH**) && (**CUDA_ARCH* < 600)
**device** double atomicAdd(double* address, double val)
{
    unsigned long long int* address_as_ull =
        (unsigned long long int*)address;

    ' '
    unsigned long long int old = *address_as_ull, assumed;

    do {
        assumed = old;
        old = atomicCAS(
            address_as_ull,
            assumed,
            __double_as_longlong(
                val + __longlong_as_double(assumed)
            )
        );
    };
}
```

```

} while (assumed != old);

return __longlong_as_double(old);
'''

}
#endif

```

This fallback implementation emulates double-precision `atomicAdd()` using `atomicCAS()` and provides compatibility with GPUs having compute capability lower than 6.0. Although slower than the native implementation, it allows the code to run correctly on older NVIDIA hardware.

Step 3: Simulation

After successful compilation, launch the parallel simulation using MPI. The execution command is specified in `runme_volta.sh`:

```

run_cmd="-np $nprocs -rf gpu_rankfile_volta
--mca pml ob1
--mca btl self,vader,tcp"

echo $mpirun $run_cmd

$mpirun $run_cmd ./a.out

```

Here, `nprocs` specifies the total number of MPI processes (GPUs), while `gpu_rankfile_volta` defines the mapping between MPI ranks and GPU devices. The executable `a.out` is the compiled CUDA-MPI solver.

During execution, each MPI process loads its corresponding subdomain and performs the wave-propagation simulation in parallel on the assigned GPU. The recorded receiver responses are written to disk for subsequent post-processing and computation of velocity dispersion and attenuation.

Step 4: Post-processing

After the simulation is completed, the dimensionless velocity and attenuation corresponding to each dimensionless parameter are computed using the post-processing routine in `main.m`:

```

%% -----
% Compute velocity and attenuation
%% -----

[Vp_fast, QQ1] = compute_Vp_Q_from_Vx( ...
fname1, fname2, nt, irx1, irx2, dt, dx, freq, false);

Q1(test_n) = abs(QQ1);
Vx_CM(test_n) = Vp_fast;

fprintf('V_nd = %.6e\n', Vp_fast);
fprintf('Q^{-1} = %.6e\n', Q1(test_n));

```

The function `compute_Vp_Q_from_Vx` calculates the dimensionless phase velocity (V_{nd}) and inverse quality factor (Q^{-1}) from the waveforms recorded at two receiver locations. The

resulting values are subsequently stored and used to construct the velocity-dispersion and attenuation curves as functions of the dimensionless parameter.

5 Output

The simulation generates the following output files and directories:

- Receiver waveforms stored as **x_Vx_rec1_num.res** and **x_Vx_rec2_num.res**, where **x** denotes the GPU index corresponding to the subdomain containing the receiver location, and **num** denotes the index of the dimensionless parameter.
- Plane-averaged wavefield snapshots saved in the directory **Saved_wavefield_figures**.
- Velocity-dispersion curves saved in the directory **Saved_wavefield_figures**.
- Attenuation curves saved in the directory **Saved_wavefield_figures**.
- Numerical results of the dimensionless velocity and attenuation stored in the directory **Saved_results**.

6 Reproducibility

All figures presented in the manuscript can be reproduced using the scripts included in this package and the parameters described in the manuscript.

The numerical results reported in the manuscript were generated using four NVIDIA V100 GPUs with 32 GB memory per GPU.

7 Contact

Zhiyu Hou

University of Lausanne

Email: zhiyu.hou@unil.ch; houzhiyu474@163.com

8 License

This code is provided solely for academic and non-commercial research purposes.

Copyright © Zhiyu Hou, Yury Alkhimenkov, and Beatriz Quintal.

All rights reserved.